

K-Nearest Neighbors (KNN)

Analítica de Datos

¿Qué es K-Nearest Neighbors?

KNN es uno de los algoritmos de machine learning más simples e intuitivos.

“Las cosas similares tienden a estar cerca”

- Algoritmo “lazy” (vago) porque no entrena un modelo explícito.
- No paramétrico (no hace suposiciones sobre la distribución de los datos)
- Sensible a la escala de las variables
- Depende del valor de k y la métrica de distancia

¿Cómo funciona KNN?

1. Llega un nuevo punto a clasificar o predecir

¿Cómo funciona KNN?

1. Llega un nuevo punto a clasificar o predecir
2. Calcula la distancia a todos los puntos de entrenamiento

¿Cómo funciona KNN?

1. Llega un nuevo punto a clasificar o predecir
2. Calcula la distancia a todos los puntos de entrenamiento
3. Selecciona los k puntos más cercanos (vecinos)

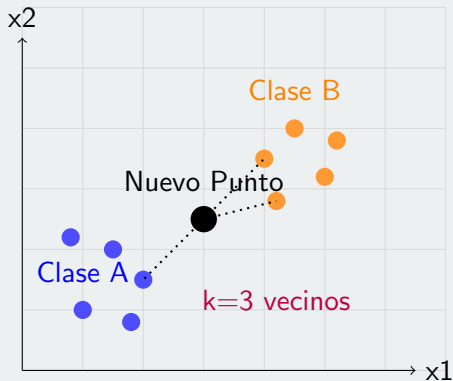
¿Cómo funciona KNN?

1. Llega un nuevo punto a clasificar o predecir
2. Calcula la distancia a todos los puntos de entrenamiento
3. Selecciona los k puntos más cercanos (vecinos)
4. Para clasificación: vota por la clase mayoritaria

¿Cómo funciona KNN?

1. Llega un nuevo punto a clasificar o predecir
2. Calcula la distancia a todos los puntos de entrenamiento
3. Selecciona los k puntos más cercanos (vecinos)
4. Para clasificación: vota por la clase mayoritaria
5. Para regresión: promedia los valores objetivo

Visualización del Concepto

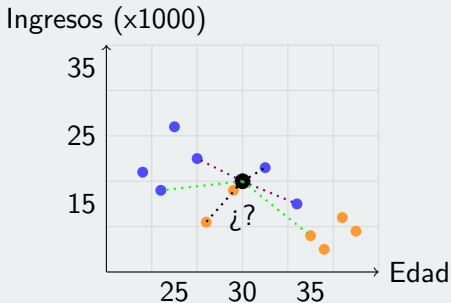


Con $k=3$: 2 puntos naranjas + 1 punto azul $\rightarrow$ Predicción: Clase B

Algoritmo:

1. Calcular distancia del nuevo punto a todos los de entrenamiento
2. Seleccionar los k puntos con menor distancia
3. Contar votos de cada clase y asignar la mayoritaria

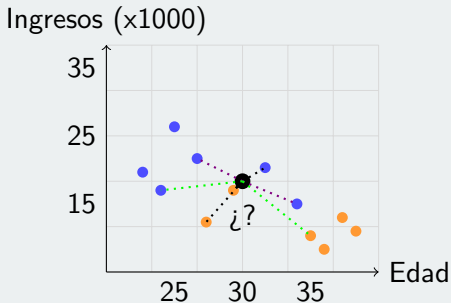
Efecto del Parámetro k



Ejemplo numérico: Aprobado (azul) vs Rechazado (naranja)

- $k=3$:
- $k=5$:
- $k=7$:

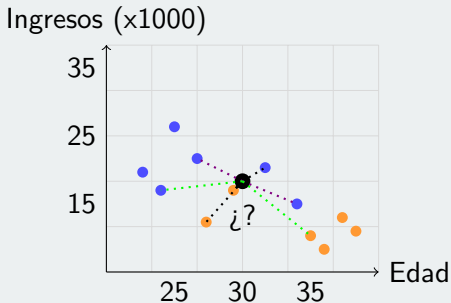
Efecto del Parámetro k



Ejemplo numérico: Aprobado (azul) vs Rechazado (naranja)

- $k=3$: 1 aprobado, 2 rechazado → **Rechazado**
- $k=5$:
- $k=7$:

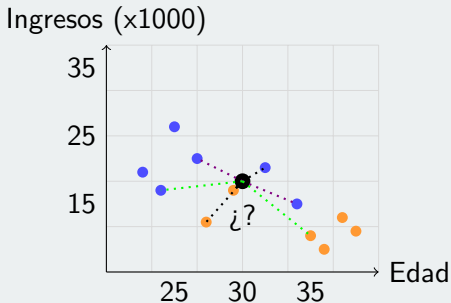
Efecto del Parámetro k



Ejemplo numérico: Aprobado (azul) vs Rechazado (naranja)

- $k=3$: 1 aprobado, 2 rechazado → **Rechazado**
- $k=5$: 3 aprobados, 2 rechazados → **Aprobado**
- $k=7$:

Efecto del Parámetro k



Ejemplo numérico: Aprobado (azul) vs Rechazado (naranja)

- $k=3$: 1 aprobado, 2 rechazado → **Rechazado**
- $k=5$: 3 aprobados, 2 rechazados → **Aprobado**
- $k=7$: 4 aprobados, 3 rechazados → **Aprobado**

Selección del Parámetro k

Reglas empíricas:

- $k = \sqrt{n}$ donde n es el número de observaciones
- Cross-validation: probar diferentes valores
- Grid search: búsqueda exhaustiva

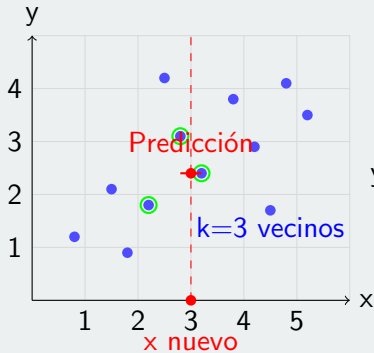
Consideraciones importantes:

- k impar evita empates en clasificación binaria
- k muy pequeño $\rightarrow$ overfitting
- k muy grande $\rightarrow$ underfitting
- Considerar el balance de clases. ¿Qué pasa si tengo 25 aprobados y 3 rechazados?

Algoritmo:

1. Calcular distancia del nuevo punto a todos los de entrenamiento
2. Seleccionar los k puntos con menor distancia
3. Promediar los valores objetivo de los k vecinos

Ejemplo Visual de Regresión



$$y = (3.1 + 2.4 + 1.8)/3 = 2.4$$

Pesos por Distancia

Hay veces en donde sería lógico darle más importancia a los vecinos más cercanos. Lo que podemos hacer ahí es en lugar de un promedio simple, usar pesos inversamente proporcionales a la distancia:

$$\hat{y} = \frac{\sum_{i=1}^k w_i \cdot y_i}{\sum_{i=1}^k w_i}$$

donde $w_i = \frac{1}{d_i}$ y d_i es la distancia al vecino i -ésimo.

¿Por qué es importante?

- Determina qué puntos considera “similares”
- Afecta la predicción final
- Es sensible a la escala de las variables
- Debe coincidir con el tipo de datos

Distancia Euclidiana

Cuándo usar: Variables continuas en la misma escala

$$d(x_1, x_2) = \sqrt{\sum_{i=1}^n (x_{1i} - x_{2i})^2}$$

Características:

- Es la distancia “real” (línea recta)
- Penaliza más las diferencias grandes (por el cuadrado)
- Sensible a outliers
- Requiere que las variables estén en la misma escala

Distancia Euclidiana

Ejemplo práctico: Predecir precio de casa

- Casa A: 100m², 3 habitaciones, precio \$150,000
- Casa B: 120m², 4 habitaciones, precio \$180,000
- Casa nueva: 105m², 3 habitaciones

Distancia a Casa A: $\sqrt{(105 - 100)^2 + (3 - 3)^2} = 5,00$

Distancia a Casa B: $\sqrt{(105 - 120)^2 + (3 - 4)^2} = 15,03$

Si k=1, elegís la más cercana (Casa A). Predicción: \$150,000

Si k=2, promediás ambas: $(\$150,000 + \$180,000)/2 = \$165,000$

Pero hay un problema: Si una variable tiene números mucho más grandes, domina todo. Si comparás ingresos (en miles) con número de hijos, los ingresos van a mandar.

Distancia Manhattan

Inspiración: Moverse en una ciudad con cuadrículas (como Manhattan). ¿Qué pasa si nos queremos mover de la 1st Ave, 2nd St a la 4th Ave, 6th St? ¿Cómo sería la distancia euclideana? ¿Es real?

$$d(x_1, x_2) = \sum_{i=1}^n |x_{1i} - x_{2i}|$$

Características:

- Suma de diferencias absolutas
- Menos sensible a outliers que Euclidiana
- Más robusta en espacios de alta dimensión
- Interpretación más intuitiva para conteos

Distancia Manhattan

Cuándo usar:

- Variables con interpretaciones diferentes
- Cantidades o conteos
- Datos con outliers (sin que dominen)

Ejemplo: Recomendación de productos

- Cliente A: 5 libros, 2 películas, 3 ropa, le gusta la ciencia ficción
- Cliente B: 3 libros, 4 películas, 1 ropa, le gusta el drama
- Cliente C: 2 libros, 1 película, 4 ropa, le gusta la ciencia ficción
- Cliente nuevo: 4 libros, 2 películas, 2 ropa, género desconocido

Distancia Manhattan

Distancia del cliente nuevo a Cliente A:

$$d(x_1, x_2) = |4 - 5| + |2 - 2| + |2 - 3| = 1 + 0 + 1 = 2$$

Distancia del cliente nuevo a Cliente B:

$$d(x_1, x_2) = |4 - 3| + |2 - 4| + |2 - 1| = 1 + 2 + 1 = 4$$

Distancia del cliente nuevo a Cliente C:

$$d(x_1, x_2) = |4 - 2| + |2 - 1| + |2 - 4| = 2 + 1 + 2 = 5$$

Si $k=1$, elegís la más cercana (Cliente A). Predicción: le gusta ciencia ficción

Si $k=3$, elegís los 3 más cercanos: Cliente A (ciencia ficción), Cliente B (drama), Cliente C (ciencia ficción) $\rightarrow$ Predicción: le gusta ciencia ficción.

Generalización de Euclidiana y Manhattan

$$d(x_1, x_2) = \left(\sum_{i=1}^n |x_{1i} - x_{2i}|^p \right)^{1/p}$$

Características:

- Parámetro p controla qué tan “exigente” es la métrica
- p pequeño: menos penalización a diferencias grandes
- p grande: más penalización a diferencias grandes
- Útil cuando querés ajustar la sensibilidad

Casos especiales:

- $p = 1$: Distancia Manhattan (suma de diferencias absolutas).
- $p = 2$: Distancia Euclidiana (la diagonal “real”).
- $p = 3, 4, 5$: A medida que p aumenta, las diferencias grandes pesan más (se penalizan más fuerte).
- $p \rightarrow \infty$: Distancia de Chebyshev. Solo considera la diferencia máxima en alguna coordenada. Ejemplo: en un proceso de fábrica en paralelo, el tiempo total lo determina la etapa más lenta (la mayor diferencia porque es el cuello de botella).

Ejemplo de Distancia de Minkowski

Notas de estudiantes (1 al 10)

- Estudiante A: 10, 7, 6, 9
- Estudiante B: 6, 8, 8, 7

Comparación con diferentes valores de p:

- Con $p=1$:
 $|10 - 6| + |7 - 8| + |6 - 8| + |9 - 7| = 4 + 1 + 2 + 2 = 9$
- Con $p=2$: $\sqrt{4^2 + 1^2 + 2^2 + 2^2} = \sqrt{16 + 1 + 4 + 4} = \sqrt{25} = 5$
- Con $p=3$:
 $\sqrt[3]{4^3 + 1^3 + 2^3 + 2^3} = \sqrt[3]{64 + 1 + 8 + 8} = \sqrt[3]{81} = 4,33$
- Con $p=7$: $\sqrt[7]{4^7 + 1^7 + 2^7 + 2^7} \approx 3,16$
- Con p muy grande (Chebyshev): $\max(4, 1, 2, 2) = 4$

Distancia de Hamming

Si los datos son categóricos como género, color, tipo de producto, etc, no podemos representar atributos como colores o estados en un eje X/Y. Hamming mide cuántas posiciones difieren y suma 1 o 0 por cada una.

$$d(x_1, x_2) = \sum_{i=1}^n I(x_{1i} \neq x_{2i})$$

Características:

- Cuenta simplemente cuántas características son diferentes
- Cada atributo distinto suma 1, los iguales suman 0
- Perfecto para datos que no son números
- Muy simple de calcular e interpretar

Cuándo usar:

- Variables categóricas (género, color, tipo)
- Datos binarios (sí/no, verdadero/falso)
- Códigos de longitud fija
- Cuando no podés representar atributos en ejes numéricos

Ejemplo de Distancia de Hamming

Clasificación de clientes basada en características categóricas

- Cliente A: M, Soltero, Empleado, **Sí** tiene tarjeta
- Cliente B: F, Casada, Empleada, **No** tiene tarjeta

Comparación posición por posición:

- M vs F $\rightarrow$ diferente (1)
- Soltero vs Casada $\rightarrow$ diferente (1)
- Empleado vs Empleada $\rightarrow$ igual (0)
- Sí tiene tarjeta vs No tiene tarjeta $\rightarrow$ diferente (1)

Distancia de Hamming = $1 + 1 + 0 + 1 = 3$

¿Cuál Métrica Elegir?

- **Solo números, misma escala** → Euclidiana.
- **Solo números, escalas diferentes** → Manhattan (o normalizar + euclidiana).
- **Solo categorías** → Hamming.
- **Mezcla de números y categorías** → Combinar métricas o convertir todo a números (con criterio).
- **Querés controlar qué tan “exigente” es** → Minkowski con p que te guste.
- **No estás seguro** → Probá Manhattan, es la más robusta.

Normalización de Variables

Problema: KNN es sensible a la escala de las variables

Ejemplo:

- Edad: 25 a 60 años
- Salario: 30,000 a 100,000 pesos

Soluciones:

- **Min-Max Scaling:** $x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}}$
- **Z-Score:** $x_{norm} = \frac{x - \mu}{\sigma}$

Resultado: Todas las variables quedan en la misma escala

Ventajas de KNN

- Fácil de entender e implementar
- No paramétrico (no asume distribuciones)
- Flexible para clasificación y regresión
- Se adapta a nuevas observaciones sin reentrenar
- Puede capturar relaciones complejas

Desventajas de KNN

- Computacionalmente pesado con muchos datos
- Muy sensible a la escala de las variables
- Afectado por ruido y outliers
- Elección de k es crucial
- No genera modelo explícito

Curse of Dimensionality

Problema: En espacios de alta dimensionalidad (por ejemplo 100 variables: 100 dimensiones), todos los puntos tienden a estar prácticamente a la misma distancia.

Consecuencias:

- Los vecinos dejan de ser realmente cercanos
- Las distancias pierden capacidad de discriminación
- Afecta el desempeño de KNN

Soluciones:

- Selección de características
- Reducción de dimensionalidad (PCA, t-SNE)
- Métricas especializadas

Implementación en Python

```
1 from sklearn.neighbors import KNeighborsClassifier, KNeighborsRegressor
2 from sklearn.preprocessing import StandardScaler # o MinMaxScaler
3 from sklearn.model_selection import cross_val_score, train_test_split
4 from sklearn.metrics import classification_report, confusion_matrix
5
6 # Preparacion de datos
7 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    random_state=42)
```

Normalización y Preparación

```
1 # Normalizacion
2 scaler = StandardScaler() # o MinMaxScaler()
3 X_train_scaled = scaler.fit_transform(X_train)
4 X_test_scaled = scaler.transform(X_test)
5
6 # Para clasificacion
7 knn_clf = KNeighborsClassifier(
8     n_neighbors=5,
9     weights='distance', # Pesos por distancia
10    metric='euclidean'
11 )
12
13 # Para regresion
14 knn_reg = KNeighborsRegressor(
15     n_neighbors=3,
16     weights='uniform',
17     metric='manhattan'
18 )
```

Selección de k con Cross-Validation

```
1 # Entrenamiento
2 knn_clf.fit(X_train_scaled, y_train)
3 knn_reg.fit(X_train_scaled, y_train)
4
5 # Predicciones
6 y_pred_clf = knn_clf.predict(X_test_scaled)
7 y_pred_reg = knn_reg.predict(X_test_scaled)
8
9 # Validacion cruzada para seleccionar k
10 k_values = range(1, 31)
11 cv_scores = []
12
13 for k in k_values:
14     knn = KNeighborsClassifier(n_neighbors=k)
15     scores = cross_val_score(knn, X_train_scaled, y_train, cv=5,
16                             scoring='accuracy')
17     cv_scores.append(scores.mean())
18
19 optimal_k = k_values[np.argmax(cv_scores)]
20 print(f"Mejor k: {optimal_k}")
```

Evaluación del Modelo

```
1 # Evaluacion de clasificacion
2 print("Reporte de clasificacion:")
3 print(classification_report(y_test, y_pred_clf))
4 print("Matriz de confusion:")
5 print(confusion_matrix(y_test, y_pred_clf))
6
7 # Evaluacion de regresion
8 mse = mean_squared_error(y_test, y_pred_reg)
9 r2 = r2_score(y_test, y_pred_reg)
10 print(f"MSE: {mse}")
11 print(f"R-squared: {r2}")
```

Clasificación

Clasificar clientes, diagnosticar
pacientes, detectar spam

Regresión

Predecir precios, estimar
temperatura, calcular demanda

Optimizaciones:

- KD-Trees o Ball Trees para búsqueda rápida
- Weighted KNN con pesos por distancia
- Filtrado de ruido y outliers
- Métricas adaptativas

¿Dudas?
¿Consultas?



Universidad de
SanAndrés